

ASCURE Admin Web Configuration Experience

Assessment of Organizations, Users, MAINHEADs, Operational Regions, Capabilities, and Site Visits.
Read-only review against Blueprint V1–V10. No code modified.

1. Capability catalogue — current state of truth

One flat table, no structure. The Capability model (`prisma/schema.prisma:459`) has only `name` / `code` / `description` / `isActive`. There is no `kind`, `category`, `scope`, or `asset-type` link column. The DTOs confirm this: `CreateCapabilityDto` accepts only those four fields.

Seeded codes (`prisma/seed.ts:267–282`) — all live in the same bag:

Code	What it semantically is
INSPECTION	Workspace / operational role
MAINTENANCE	Workspace / operational role
QA_VALIDATION	Governance role
REPORTING	Operational role
SAVR	Asset / inspection domain
SAVT	Asset / inspection domain
PENCAWANG	Asset / inspection domain
FEEDER_PILLAR	Asset / inspection domain
LINK_BOX	Asset / inspection domain
CABLE_BRIDGE	Asset / inspection domain
UNDERGROUND_CABLE	Asset / inspection domain
THERMAL_INSPECTION	Asset / inspection domain

Parallel legacy enum. `OrganizationCapabilityType` (`schema.prisma:215`) is a separate fixed enum (`SURVEY` / `INSPECTION` / `MAINTENANCE` / `REPAIR` / `CIVIL` / `DISTRIBUTION` / `THIRTY_THREE_KV` / `EMERGENCY_RESPONSE` / `QA_VALIDATION` / `REPORTING` / `OTHER`) used only by `OrganizationCapability` — overlapping but inconsistent vocabulary with the `Capability` table the admin UI actually edits.

Admin UI shape. `enterprise-list-client.tsx:480–517` renders capabilities as a single flat checkbox grid (`CapabilityPicker`), and `users-client.tsx:230–276` does the same (`UserCapabilityPicker`). No grouping, no separator, no helper text.

2. Findings — by question

2.1 Capability clarity

- The label set mixes nouns from different ontologies (an asset class like SAVR sits next to a workspace like INSPECTION). An admin reading SAVR + INSPECTION + REPORTING cannot tell from the UI alone what the user/team will or won't be able to do.
- Codes are upper-snake-case and shown verbatim (`optionLabel` prefixes the code). Good for engineers, opaque for the operational admins Blueprint V6/V35 says we should design for.
- No description is shown in the picker (it exists on the model and the create form, but the picker checkbox only renders code - name). The single column of information density the admin needs ("what does ticking this turn on?") is invisible at pick time.
- THERMAL_INSPECTION overlaps INSPECTION semantically; MAINTENANCE overlaps the operational workspace term used in V10 Mobile Workspaces. Same word, two meanings (catalogue capability vs. mobile workspace) — guaranteed misread.

2.2 Capability categorisation

- There is none. Per Blueprint V4 §34–35, capabilities are envisaged as a system that gates "operational visibility and assignment" across Org/Branch/MAINHEAD/Team/User — but no taxonomy is encoded.
- Practically, two categories already exist in the codes themselves: **workspace/operational** (INSPECTION, MAINTENANCE, QA_VALIDATION, REPORTING) and **domain/asset** (SAVR, SAVT, PENCAWANG, FEEDER_PILLAR, LINK_BOX, CABLE_BRIDGE, UNDERGROUND_CABLE, THERMAL_INSPECTION). The seed file's separation of `OrganizationCapabilityType` (workspace-ish) from `Capability` (domain-ish) hints at the intended split, but the unified `Capability` table flattens that back.

2.3 Workspace capabilities vs operational capabilities

- The mobile workspaces are exactly two: **Inspection** and **Maintenance** (V10 §Mobile Workspaces; V3 §3–4, §13). Mobile workspace access is governed by capability presence (commit 965e68d: "Make mobile workspace access capability-driven"). So INSPECTION and MAINTENANCE capabilities are **workspace gates**, not operational scope.
- QA_VALIDATION and REPORTING are governance roles per V6/V8 (QA belongs to ASCURA, cross-region, cross-MAINHEAD). They are **role-like capabilities**, not workspace gates and not asset gates.
- The asset codes (SAVR...THERMAL_INSPECTION) are **scoped to what assets/templates a workspace can act on**. `AssetType.capabilityId` and `InspectionTemplate.capability` already link assets and templates to a capability — so these codes really do act as **asset-class filters**, distinct from the workspace gates.
- Today the admin has to know all of this implicitly. Nothing in the UI says "ticking SAVR makes SAVR asset types visible" vs "ticking INSPECTION lets this team open the Inspection workspace."

2.4 Organization setup UX

- Organization modal is just Name / Code / Type / Active + Capabilities checkbox grid.
- Type list defaults to a placeholder enum (ASCURE / TNB / SUBCONTRACTOR / CONSULTANT / CLIENT / OTHER) — diverges from V4 §4 (GOVERNANCE / UTILITY_OWNER / INSPECTION_CONTRACTOR / MAINTENANCE_CONTRACTOR / GENERAL). The two type vocabularies don't agree.

- No way to set or visualise hierarchy from create: no branch preview, no MAINHEAD count, no contractor-vs-utility hint based on type. The form doesn't encode the principle from V6 §12 (contractor independence) or V4 §4 (organization types).

2.5 User setup UX

- User modal is one long monolithic vertical form: Name, Email, Role, Organization, Branch, Team, MAINHEAD Access (multi), Region Access (multi), Capabilities (multi), Password.
- Role list is ADMIN / MANAGER / SUPERVISOR / TECHNICIAN / VIEWER / CLIENT. The Blueprint V8 §3 personas are richer (QA Supervisor / QA Inspector / Team Leader / Maintenance Technician). Personas like QA are not selectable as a role — they currently must be assembled from capabilities + region access + organization, with no help.
- The legacy mainheadId foreign key is silently maintained: ticking the first item in MAINHEAD Access auto-writes mainheadId. Hidden coupling; the legacy single-MAINHEAD field is still there underneath G1.
- “Region Access” and “MAINHEAD Access” are independent multi-selects. Per V10 §MAINHEAD Visibility Algorithm and V8 §13, region access **inherits** all of its MAINHEADs — but the UI presents them as parallel choices with no resolution hint, so an admin can over-grant (region + every MAINHEAD individually) or under-grant (region only, then unticking a MAINHEAD that was never explicit).
- Password is set in-line on Create only. Reset requires a second modal. Fine, but no “force reset on next login,” no expiry, no copy-to-clipboard generated password.
- Capabilities again rendered as one flat checkbox grid (same UserCapabilityPicker), no hint that ticking SAVR without ticking INSPECTION yields a user that can see SAVR assets but can't enter the Inspection workspace.

2.6 MAINHEAD setup UX

- MAINHEAD modal leads with Region (Operational Region dropdown) + Legacy Branch dropdown, then a collapsed “**Legacy branch fields**” accordion (Organization / Branch name / Branch code / Region text). All of that legacy slot is still **mutable** on the form.
- Per V10 (Governance G1 mandatory) and V6 §12 (contractor independence), MAINHEAD belongs under OperationalRegion. The “Legacy Branch” select and the legacy-branch accordion are pre-G1 cruft. They are not flagged as read-only or deprecated.
- Capability picker appears here too. There's no guidance that the MAINHEAD's capabilities act as **operational scope advertised by the area** (capability resolution surfaces in MainheadCapability, schema.prisma:628).
- No way to preview which Users/Teams have access to this MAINHEAD, nor which Projects/Work Packages belong to it, from the create/edit modal. Detail page is referenced (hasDetail: true) but the modal is the primary configuration surface.

2.7 Site Visit setup UX

- Site Visit create form shows three controls relevant here: a MAINHEAD dropdown (whose empty option literally reads “**Text MAINHEAD only**”), a free-text “**MAINHEAD Text**” input below it, and a Project / Work Package row.
- The API DTO (create-site-visit.dto.ts:46, 107) accepts both mainheadId?: string (UUID) and mainhead?: string (free text). Validation in site-visits.service.ts:1104 requires **either**:

```
if (!dto.mainhead && !dto.mainheadId) missingFields.push('MAINHEAD');
```

- So today a visit can ship with a free-text MAINHEAD and **no** structured FK.
- The detail view (`site-visit-detail-client.tsx:85-92`) falls back: `mainheadRecord?.name` → `mainheadRecord?.code` → `mainhead (text)` → "Not recorded".

3. Should INSPECTION / MAINTENANCE / QA_VALIDATION / REPORTING be presented differently from SAVR / SAVT / PENCAWANG / FEEDER_PILLAR / LINK_BOX / CABLE_BRIDGE / UNDERGROUND_CABLE / THERMAL_INSPECTION?

Yes. They are two different concepts wearing the same checkbox.

Group	Real role	Today
INSPECTION, MAINTENANCE	Mobile workspace gates (V10 Mobile Workspaces, common to all)	Same flat checkbox
QA_VALIDATION, REPORTING	Cross-cutting role / authority capabilities (V6 §8, V8 §56)	Same flat checkbox
SAVR, SAVT, PENCAWANG, FEEDER_PILLAR, LINK_BOX, CABLE_BRIDGE, UNDERGROUND_CABLE, THERMAL_INSPECTION	Asset class / inspection domain scope (V10 §58, V10 §59, V10 §60)	Same flat checkbox

Presenting them in one undifferentiated list pushes admin error onto every Org / Branch / MAINHEAD / Team / User assignment, and makes the most common mistake invisible: granting a domain (SAVR) without the workspace (INSPECTION), or the reverse.

4. Is the MAINHEAD text fallback still required after Governance G1?

No. Governance G1 (V10 §Governance G1, V8 §12, V4 §6–7, V6 §13) makes Mainhead a first-class entity with FK references everywhere (UserMainheadAccess, MainheadCapability, Project.mainheadId, WorkPackage.mainheadId, SiteVisit.mainheadId, InspectionTemplate.mainheadId, OperationalSession.mainheadId). The text column on SiteVisit (mainhead String?, schema.prisma:927) and the parallel admin-side “MAINHEAD Text” input are pre-G1 fallbacks.

Concrete harms of keeping it:

- MAINHEAD visibility algorithm bypass** — UserMainheadAccess + UserOperationalRegionAccess (V10 §MAINHEAD Visibility Algorithm) gate by mainheadId. A visit with text-only MAINHEAD has no FK to gate against. Whose queue does it land in?
- Template resolution bypass** — Resolution order MAINHEAD → REGION → BRANCH → ORG → GLOBAL (V10 §Template Governance) keys off mainheadId. Text MAINHEAD silently falls to GLOBAL templates only.
- Capability resolution bypass** — MainheadCapability rows can’t apply to a text MAINHEAD; site visits scoped only by string get no asset-class scoping.
- Reporting drift** — “Klang Utama,” “KLANG_UTAMA,” “Klang Utama-1” all coexist as text variants and won’t roll up.
- Governance audit gap** — Per V6 §9 / V8 §25 MAINHEAD is the operational area entity that change-control is tracked against. A text-only visit has no entity to audit.

There is no operational scenario in V10 where a MAINHEAD that matters isn’t already an admin-configurable record. The text fallback should be retired.

5. UX concerns (priority order)

1. **Flat capability list mixes workspaces, governance roles, and asset domains.** Same checkbox for INSPECTION and LINK_BOX. Highest-frequency admin error surface.
2. **No descriptions surfaced at pick time.** Description exists on the model but the picker only renders code - name.
3. **“MAINHEAD Text” input + “Text MAINHEAD only” dropdown placeholder** — invites the operations team to bypass G1.
4. **Two parallel capability vocabularies** (Capability table vs OrganizationCapabilityType enum) with overlapping but mismatched terms.
5. **Two parallel Organization type vocabularies** (UI default ASCURE/TNB/SUBCONTRACTOR/CONSULTANT/CLIENT/OTHER vs Blueprint GOVERNANCE/UTILITY_OWNER/INSPECTION_CONTRACTOR/MAINTENANCE_CONTRACTOR/GENERAL).
6. **User role list missing QA personas.** No QA_SUPERVISOR / QA_INSPECTOR role; QA must be reconstructed from org + capability + region access.
7. **MAINHEAD form still surfaces editable “Legacy Branch” + accordion** post-G1, with no deprecation hint.
8. **Region Access vs MAINHEAD Access** have no resolution hint — admin can’t tell what the user will actually see.
9. **Silent mainheadId write from MAINHEAD Access[0]** (users-client.tsx:477) hides a load-bearing legacy field.
10. **No preview of impact** anywhere (e.g., creating a MAINHEAD doesn’t show which Region/Branch/users/teams it inherits to).

6. Recommended improvements (no code changes — design only)

A. Split the capability catalogue into three sections in the picker

Without changing the data model, present capabilities in **grouped fieldsets** in the picker. The classification can be code-prefix or a derived map until a kind column is added:

Workspace access

- Inspection Lets users enter the Inspection workspace.
- Maintenance Lets users enter the Maintenance workspace.

Governance & reporting

- QA Validation Cross-MAINHEAD verification authority.
- Reporting Operational reporting and export.

Asset domains

- SAVR Pole inspections (Sesalur Atas Voltan Rendah)
- SAVT Route inspections
- Pencawang Substation inspections
- Feeder Pillar
- Link Box
- Cable Bridge
- Underground Cable
- Thermal Inspection

Add description text under each checkbox (it's already on the model). Show a one-line "what this turns on" caption per group.

B. Add a real kind column to Capability

Once the grouping above is validated, persist it. Suggested values: `WORKSPACE`, `GOVERNANCE`, `ASSET_DOMAIN`. Backfill from current codes deterministically.

C. Retire the MAINHEAD text fallback

- Make MAINHEAD dropdown **required** in the Site Visit form. Drop the empty option labelled "Text MAINHEAD only."
- Hide the MAINHEAD Text input from the create form. Display the read-only legacy value only on the detail view when `mainheadRecord` is null and the visit predates G1, marked as **Legacy**.
- API: change the `if (!dto.mainhead && !dto.mainheadId)` validator to require `mainheadId`; keep `mainhead` as a read-only string the API may return but not accept on create.
- Migration: backfill `mainheadId` for existing visits by matching `mainhead` text (case-insensitive) against `Mainhead.name / Mainhead.code`. Park unmatched visits in a "MAINHEAD reconciliation" queue.

D. Organisation type list — adopt blueprint vocabulary

Replace the placeholder enum with `GOVERNANCE / UTILITY_OWNER / INSPECTION_CONTRACTOR / MAINTENANCE_CONTRACTOR / GENERAL`. This eliminates one of two competing vocabularies.

E. User personas — surface QA roles directly

Add `QA_SUPERVISOR` and `QA_INSPECTOR` to the role list (V8 §5–6). Their selection should pre-fill `region access = all` and `capabilities = {QA_VALIDATION, REPORTING}` as a starting point, leaving the admin free to adjust. Optional: introduce `TEAM_LEADER` and `MAINTENANCE_TECHNICIAN` per V8 §8, §10.

F. MAINHEAD form — mark legacy fields read-only

Move "Legacy Branch" select and the "Legacy branch fields" accordion under a read-only "Pre-G1 references" section. Keep the data visible for historical visits; block edits. Lead with `OperationalRegion + Capabilities` as the canonical fields.

G. Region vs MAINHEAD access — show what's resolved

Below the two pickers, render a small computed line: "Effective MAINHEAD access: KL Timur, KL Barat, Subang (via Region: Klang Valley) + Bentong (direct)." Uses the same logic as `GET /users/me/mainheads`. Stops over-/under-granting.

H. Drop the silent mainheadId mirror

Stop auto-writing `user.mainheadId` from the first MAINHEAD-access checkbox. Either let it remain null for new G1 users or make it explicit ("Primary MAINHEAD" selector that the user sees).

7. Pilot impact

The pilot is SAVR-driven (V10 §SAVR Domain, V9). The capability picker today **does not block** the pilot — every required capability exists in the seed, and the admin can tick SAVR + INSPECTION + REPORTING for an inspection contractor and MAINTENANCE for the repair contractor. But:

- **Workspace gating is fragile.** If an admin ticks SAVR without INSPECTION, the user sees no workspace. If they tick INSPECTION without SAVR, the user can enter the workspace but sees no SAVR assets. There is no warning. For a pilot operator setting up dozens of accounts, this is the most likely cause of “the technician logged in and nothing was there.”
- **MAINHEAD text fallback is high risk in the pilot.** The pilot ramp-up will create visits at speed. Any admin who skips the MAINHEAD dropdown (the empty option still works) creates an audit-degraded visit. Visibility, template resolution, and reporting all silently degrade — these will only surface as “why isn’t this visit showing up on QA’s board” or “why is this defect using the wrong template.”
- **QA persona absence.** ASCURA QA users today need ADMIN role to get the cross-MAINHEAD visibility V8 §15 demands, or rely on the QA override branch in the visibility algorithm (V10 §MAINHEAD Visibility Algorithm step 7). Either is fine functionally but requires the pilot admin to know which lever to pull.

Pilot-grade fix order:

1. Hide / require MAINHEAD dropdown on Site Visit create.
2. Group the capability picker visually (no schema change required).
3. Document for the pilot admin: which capability combinations produce which workspaces.

Everything else can wait past the pilot.

8. Migration impact

Change	Schema migration	Data migration	Risk
Group capabilities in picker	None	None	None
Add Capability.kind enum	Additive column, default ASSET to DEMAND	UPDATE to set workspace/governance kinds	Low
Retire MAINHEAD text fallback (UI)	None	None	None — UI-only
Retire MAINHEAD text fallback (API)	Make SiteVisit.mainheadId non-nullable	Backfill on SiteVisit by mainheadId	Medium — API-only, not the visits we have in prod
Replace org type enum	Migrate OrganizationType enum	Map org type, SOBBENT to CAC, FROTH to FROTH, etc.	Low
Collapse OrganizationCapabilityType enum	Drop type, replace CapabilityOrganization	CapAbleType table, migrate live rows to OrganizationCapabilityAssignment	Low
Add QA persona roles	Additive enum values	None for existing users	Low
Mark MAINHEAD “Legacy Branch” (UI)	None (UI only)	LegacyBranchId if not already	Low

The blockers for pilot are A and C (capability grouping + MAINHEAD text retirement at the UI layer) — both are UI-only and add zero risk to production data.

Summary

The configuration surface today works but treats different governance concepts as the same checkbox, and still ships a pre-G1 free-text MAINHEAD path that quietly defeats the Governance G1 algorithms the rest of the platform is built around. The highest-leverage moves are UI-only: split capabilities into **Workspace / Governance / Asset Domain** groups in the existing pickers, and remove the free-text MAINHEAD route from Site Visit creation. Schema cleanups (kind column, enum reconciliation, text-column retirement) can follow the pilot. No code was modified in producing this assessment.